VERİ ANALİSTİ MÜFREDATI 2026

MODÜL 7
SQL İLERİ

CTE • Window Functions • ROW_NUMBER • RANK • LAG/LEAD • Date Functions • Cohort • Funnel

Hedef: Orta seviye SQL bilen bir analistin, gerçek iş problemlerinde satır bazlı karşılaştırma, sıralama, zaman analizi, cohort ve funnel sorgularını güvenilir ve okunabilir biçimde yazabilmesi.

Bu modül PostgreSQL ağırlıklı örnekler kullanır. MySQL, SQL Server, BigQuery ve Snowflake üzerinde fonksiyon isimleri veya tarih sözdizimi küçük farklılıklar gösterebilir.

1. SQL İleri Seviyeye Geçiş: Ne Değişiyor?

Temel ve orta SQL'de çoğunlukla filtreleme, toplulaştırma ve tabloları birleştirme yapılır. İleri SQL'de ise asıl hedef aynı veri seti içinde satırlar arası ilişki kurmak, geçmiş ve gelecek satırlara bakmak, gruplar içinde sıralama yapmak ve analitik hesapları veri tabanında gerçekleştirmektir.

İleri SQL'in ana problem sınıfları

- Her müşteri için ilk ve son siparişi bulmak
- Her kategori içinde en yüksek satış yapan ilk 3 ürünü bulmak
- Bu ayki geliri geçen ayla karşılaştırmak
- Kullanıcıların kayıt oldukları aya göre sonraki aylarda geri dönüp dönmediklerini ölçmek
- Bir funnel içinde kullanıcıların hangi adımda düştüğünü hesaplamak
- Uzun ve karmaşık sorguları okunabilir parçalara bölmek

> Analist bakış açısı: İleri SQL, daha uzun sorgu yazmak değil; karmaşık iş sorusunu doğru grain, doğru partition ve doğru zaman mantığıyla çözmektir.

Örnek veri modeli

Tablo	Grain	Önemli kolonlar
customers	1 satır = 1 müşteri	customer_id, signup_date, country
orders	1 satır = 1 sipariş	order_id, customer_id, order_date, status, revenue
order_items	1 satır = 1 sipariş kalemi	order_id, product_id, quantity, unit_price
events	1 satır = 1 kullanıcı olayı	user_id, event_time, event_name, session_id

2. CTE - Common Table Expression

CTE, bir sorgunun ara sonucuna isim vererek ana sorguyu daha okunabilir hale getirir. WITH anahtar sözcüğüyle başlar. Bir CTE geçici tablo değildir; çoğu sistemde yalnızca o SQL ifadesinin yaşam süresi boyunca vardır.

```
WITH monthly_sales AS (
  SELECT DATE_TRUNC('month', order_date) AS month,
         SUM(revenue) AS revenue
  FROM orders
  WHERE status = 'completed'
  GROUP BY 1
)
SELECT *
FROM monthly_sales
ORDER BY month;
```

Neden CTE kullanılır?

- Karmaşık sorguyu mantıksal adımlara ayırmak
- Aynı ara sonucu sonraki aşamada anlamlı bir isimle kullanmak
- Debug sürecini kolaylaştırmak
- Cohort ve funnel gibi çok aşamalı analizlerde iş mantığını görünür hale getirmek

Birden fazla CTE

```
WITH customer_sales AS (
  SELECT customer_id, SUM(revenue) AS total_revenue
  FROM orders
  WHERE status = 'completed'
  GROUP BY customer_id
),
segments AS (
  SELECT customer_id, total_revenue,
         CASE WHEN total_revenue >= 5000 THEN 'High'
              WHEN total_revenue >= 1000 THEN 'Mid'
              ELSE 'Low' END AS segment
  FROM customer_sales
)
SELECT segment, COUNT(*) AS customers
FROM segments
GROUP BY segment;
```

CTE performans notu: CTE otomatik olarak daha hızlı demek değildir. Optimizasyon davranışı veritabanına göre değişir. CTE'yi öncelikle doğruluk ve okunabilirlik için kullan; performansı execution plan ile ölç.

3. Window Functions: İleri SQL’in Kalbi

Window function, GROUP BY gibi satırları tek satıra indirmeden, bir satırın çevresindeki belirli satır kümesi üzerinde hesaplama yapar. Bu nedenle hem detay satırlarını korur hem de toplam, sıra, önceki değer veya hareketli ortalama gibi analitik bilgileri aynı sonuçta üretebilir.

```
function(...) OVER (
  PARTITION BY ...
  ORDER BY ...
  ROWS BETWEEN ...
)
```

GROUP BY ile farkı

Özellik	GROUP BY	Window Function
Satır sayısı	Gruplar kadar satıra düşürür	Detay satırlarını korur
Toplam üretir mi?	Evet	Evet
Sıralama/rank	Dolaylı	Doğrudan
Önceki/sonraki satır	Hayır	LAG/LEAD ile evet
Cohort / running total	Zor	Doğal kullanım alanı

PARTITION BY nedir?

PARTITION BY, hesaplamanın hangi gruplar içinde yeniden başlayacağını belirler. Örneğin her müşteri kendi pencere grubunu oluşturabilir.

```
SELECT customer_id, order_id, order_date, revenue,
       SUM(revenue) OVER (PARTITION BY customer_id) AS customer_total
FROM orders;
```

ORDER BY neden kritik?

Window içindeki ORDER BY, satırların analitik sırasını belirler. Özellikle ROW_NUMBER, RANK, LAG, LEAD ve running total için yanlış ORDER BY doğrudan yanlış iş sonucuna yol açar.

4. ROW_NUMBER: Her Gruba Benzersiz Sıra Numarası

ROW_NUMBER(), partition içinde 1'den başlayan benzersiz bir sıra üretir. Eşit değerler olsa bile her satıra farklı numara verir.

Her müşterinin en son siparişi

```
WITH ranked AS (
  SELECT o.*,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id
           ORDER BY order_date DESC, order_id DESC
         ) AS rn
  FROM orders o
)
SELECT *
FROM ranked
WHERE rn = 1;
```

Tie-breaker ekle: Sadece order_date ile sıralarsan aynı timestamp'e sahip iki kayıt için sonuç deterministik olmayabilir. order_id gibi ikinci bir kriter eklemek güvenli pratiktir.

Duplicate temizleme deseni

```
WITH d AS (
  SELECT *,
         ROW_NUMBER() OVER (
           PARTITION BY customer_id, order_date, revenue
           ORDER BY created_at DESC
         ) AS rn
  FROM raw_orders
)
SELECT * FROM d WHERE rn = 1;
```

Bu desen, aynı iş anahtarına sahip tekrar kayıtlar içinde hangi kaydın tutulacağını açıkça tanımlar. Ancak business key yanlış seçilirse gerçek işlemleri de silebilirsin.

Top-N per group

```
WITH p AS (
  SELECT category, product_id, SUM(revenue) AS sales,
         ROW_NUMBER() OVER (
           PARTITION BY category
           ORDER BY SUM(revenue) DESC
         ) AS rn
  FROM sales
  GROUP BY category, product_id
)
SELECT * FROM p WHERE rn <= 3;
```

5. RANK ve DENSE_RANK

RANK ve DENSE_RANK eşit değerlere aynı sırayı verir. Aralarındaki temel fark, eşitlikten sonraki sıra numarasının davranışıdır.

Satış	RANK	DENSE_RANK
1000	1	1
1000	1	1
900	3	2
800	4	3

Ne zaman hangisi?

- ROW_NUMBER: Tek bir kazanan veya benzersiz sıra gerektiğinde
- RANK: Yarışma sıralaması gibi eşitliğin sonraki sırada boşluk yaratması kabul edildiğinde
- DENSE_RANK: Eşitlikleri koruyup sıraları kesintisiz tutmak istediğinde

```
SELECT product_id, revenue,
       RANK() OVER (ORDER BY revenue DESC) AS sales_rank,
       DENSE_RANK() OVER (ORDER BY revenue DESC) AS dense_rank
FROM product_sales;
```

Analitik vaka: kategori bazında ilk 3

```
SELECT *
FROM (
  SELECT category, product_id, revenue,
         DENSE_RANK() OVER (
           PARTITION BY category
           ORDER BY revenue DESC
         ) AS rnk
  FROM product_sales
) x
WHERE rnk <= 3;
```

6. LAG ve LEAD: Satırlar Arası Zaman Karşılaştırması

LAG önceki satırdaki, LEAD sonraki satırdaki değeri getirir. Özellikle zaman serilerinde büyüme, churn sinyali ve ardışık olay analizi için kullanılır.

Aylık büyüme

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS month,
         SUM(revenue) AS revenue
  FROM orders
  WHERE status='completed'
  GROUP BY 1
), x AS (
  SELECT month, revenue,
         LAG(revenue) OVER (ORDER BY month) AS prev_revenue
  FROM m
)
SELECT month, revenue, prev_revenue,
       ROUND(100.0*(revenue-prev_revenue)/NULLIF(prev_revenue,0),2) AS mom_growth_pct
FROM x;
```

Siparişler arası gün farkı

```
SELECT customer_id, order_date,
       LAG(order_date) OVER (
         PARTITION BY customer_id ORDER BY order_date
       ) AS prev_order_date,
       order_date - LAG(order_date) OVER (
         PARTITION BY customer_id ORDER BY order_date
       ) AS days_since_prev
FROM orders;
```

LEAD ile bir sonraki olay

```
SELECT user_id, event_time, event_name,
       LEAD(event_name) OVER (
         PARTITION BY user_id ORDER BY event_time
       ) AS next_event
FROM events;
```

Önemli: LAG/LEAD fiziksel tablo sırasına değil, OVER içindeki ORDER BY sırasına göre çalışır.

7. Running Total, Moving Average ve Window Frame

Window frame, mevcut satır için pencereye hangi komşu satırların dahil edileceğini belirler. Özellikle kümülatif toplam ve hareketli ortalamalarda kritik önemdedir.

Kümülatif gelir

```
SELECT order_date, daily_revenue,
       SUM(daily_revenue) OVER (
         ORDER BY order_date
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS cumulative_revenue
FROM daily_sales;
```

7 günlük hareketli ortalama

```
SELECT order_date, daily_revenue,
       AVG(daily_revenue) OVER (
         ORDER BY order_date
         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
       ) AS ma_7d
FROM daily_sales;
```

ROWS fiziksel satır sayısına göre pencere kurar. Tarihlerde eksik günler varsa “7 satır” her zaman “7 takvim günü” anlamına gelmez. Bu fark gerçek zaman serisi analizinde önemlidir.

> Kalite kontrolü: Hareketli ortalama yazmadan önce veri setinde her gün için gerçekten bir satır olup olmadığını kontrol et.

8. Date Functions: Zaman Boyutunu Doğru Modellemek

Tarih fonksiyonları analitik SQL'in temelidir. Aylık trend, haftalık aktivite, cohort, retention ve funnel zaman kısıtları tarih işlemlerine dayanır.

Yaygın PostgreSQL örnekleri

Amaç	Örnek
Ay başlangıcına kırpma	DATE_TRUNC('month', order_date)
Yıl çıkarma	EXTRACT(YEAR FROM order_date)
Gün farkı	date_b - date_a
Aralık ekleme	order_date + INTERVAL '7 day'
Bugün	CURRENT_DATE
Timestamp -> date	event_time::date

Ay bazlı karşılaştırmada dikkat

Aylar farklı gün sayısına sahiptir. Ay ortasında rapor alıyorsan tam ayı kısmi ayla kıyaslamak yanıltıcı olabilir. Gerekirse MTD (month-to-date) karşılaştırması yap.

```
WHERE order_date >= DATE_TRUNC('month', CURRENT_DATE)
  AND order_date <  CURRENT_DATE + INTERVAL '1 day'
```

Zaman dilimi problemi

Event verisi UTC tutulup rapor yerel saatle okunuyorsa gün ve ay sınırları değişebilir. Funnel ve günlük aktif kullanıcı analizlerinde timezone dönüşümü iş tanımının parçasıdır.

9. Cohort Analizi - Mantık

Cohort analizi, kullanıcıları ortak bir başlangıç olayına göre gruplar ve sonraki dönemlerde davranışlarını izler. En klasik örnek: müşteriyi ilk satın alma ayına göre cohort'a atamak ve sonraki aylarda tekrar satın alma oranını ölçmek.

Adım 1 - İlk aktiviteyi bul

```
WITH first_order AS (
  SELECT customer_id,
         DATE_TRUNC('month', MIN(order_date)) AS cohort_month
  FROM orders
  WHERE status='completed'
  GROUP BY customer_id
)
SELECT * FROM first_order;
```

Adım 2 - Her aktivitenin dönemini belirle

```
WITH first_order AS (...),
activity AS (
  SELECT DISTINCT o.customer_id,
         f.cohort_month,
         DATE_TRUNC('month', o.order_date) AS activity_month
  FROM orders o
  JOIN first_order f USING(customer_id)
  WHERE o.status='completed'
)
SELECT * FROM activity;
```

Adım 3 - Cohort age

```
EXTRACT(YEAR FROM AGE(activity_month, cohort_month))*12
+ EXTRACT(MONTH FROM AGE(activity_month, cohort_month)) AS month_number
```

month_number = 0 kullanıcının cohort olduğu ayı, 1 bir sonraki ayı, 2 iki ay sonrasını gösterir.

10. Cohort Retention Sorgusu

```
WITH first_order AS (
  SELECT customer_id, DATE_TRUNC('month', MIN(order_date)) AS cohort_month
  FROM orders
  WHERE status='completed'
  GROUP BY customer_id
),
activity AS (
  SELECT DISTINCT o.customer_id, f.cohort_month,
         DATE_TRUNC('month', o.order_date) AS activity_month
  FROM orders o
  JOIN first_order f USING(customer_id)
  WHERE o.status='completed'
),
cohort_activity AS (
  SELECT customer_id, cohort_month,
         (EXTRACT(YEAR FROM AGE(activity_month, cohort_month))*12 +
          EXTRACT(MONTH FROM AGE(activity_month, cohort_month)))::int AS month_no
  FROM activity
),
counts AS (
  SELECT cohort_month, month_no, COUNT(DISTINCT customer_id) AS active_customers
  FROM cohort_activity
  GROUP BY 1,2
),
sizes AS (
  SELECT cohort_month, active_customers AS cohort_size
  FROM counts WHERE month_no=0
)
SELECT c.cohort_month, c.month_no, c.active_customers, s.cohort_size,
       ROUND(100.0*c.active_customers/s.cohort_size,2) AS retention_pct
FROM counts c
JOIN sizes s USING(cohort_month)
ORDER BY 1,2;
```

Cohort analizinde sık hata

- COUNT(*) kullanıp kullanıcıyı aynı ay içindeki birden fazla sipariş nedeniyle birden fazla saymak
- İlk siparişi iptal edilmiş siparişlerden de hesaplamak
- Cohort başlangıcını signup, first order ve first active event arasında net tanımlamamak
- Henüz tamamlanmamış gelecekteki cohort dönemlerini 0 retention gibi yorumlamak

İş tanımı önce gelir: Retention'ın tanımı şirkete göre değişebilir: tekrar sipariş, uygulama açma, aboneliğin devamı veya belirli bir "core action". SQL'den önce retained user tanımını yaz.

11. Funnel Analizi - Temel Mantık

Funnel, kullanıcıların ardışık iş adımlarında kaçının ilerlediğini ölçer. Örnek: product_view → add_to_cart → checkout → purchase.

Basit conditional aggregation yaklaşımı

```
SELECT
  COUNT(DISTINCT CASE WHEN event_name='product_view' THEN user_id END) AS viewers,
  COUNT(DISTINCT CASE WHEN event_name='add_to_cart' THEN user_id END) AS adders,
  COUNT(DISTINCT CASE WHEN event_name='checkout' THEN user_id END) AS checkouts,
  COUNT(DISTINCT CASE WHEN event_name='purchase' THEN user_id END) AS buyers
FROM events;
```

Bu sorgu kaba bir funnel verir; ancak olay sırasını ve süre kısıtını garanti etmez. Kullanıcı purchase yaptı diye gerçekten product_view'dan sonra yaptığı varsayılamaz.

Gerçek funnel için gerekenler

- Aynı kullanıcı veya session kimliği
- Adımların doğru kronolojik sırada gerçekleşmesi
- Gerekirse aynı session içinde olma şartı
- Adımlar arasında maksimum süre tanımı
- Tekrarlanan eventlerin nasıl ele alınacağı

12. Sıralı Funnel Sorgusu

```
WITH per_user AS (
  SELECT user_id,
    MIN(CASE WHEN event_name='product_view' THEN event_time END) AS view_time,
    MIN(CASE WHEN event_name='add_to_cart' THEN event_time END) AS cart_time,
    MIN(CASE WHEN event_name='checkout' THEN event_time END) AS checkout_time,
    MIN(CASE WHEN event_name='purchase' THEN event_time END) AS purchase_time
  FROM events
  GROUP BY user_id
)
SELECT
  COUNT(*) FILTER (WHERE view_time IS NOT NULL) AS step1,
  COUNT(*) FILTER (WHERE cart_time > view_time) AS step2,
  COUNT(*) FILTER (WHERE checkout_time > cart_time AND cart_time > view_time) AS step3,
  COUNT(*) FILTER (WHERE purchase_time > checkout_time
                   AND checkout_time > cart_time
                   AND cart_time > view_time) AS step4
FROM per_user;
```

Bu örnek mantığı öğretir ancak çoklu session ve birden fazla funnel denemesi olan kullanıcılar için daha sağlam event-sequence yaklaşımı gerekebilir.

Step conversion hesapları

```
view_to_cart = step2 / step1
cart_to_checkout = step3 / step2
checkout_to_purchase = step4 / step3
overall_conversion = step4 / step1
```

Root-cause: Funnel'da sadece en büyük mutlak kayba değil, adım dönüşüm oranına ve hacme birlikte bak. %10 düşüş 1 milyon kullanıcıda, %50 düşüş 500 kullanıcıda olduğundan daha büyük ticari etki yaratabilir.

13. Gerçek Vaka: Gelir Neden Düştü?

Senaryo: Ağustos gelirinin Temmuz'a göre %12 düştüğü bildiriliyor. İleri SQL ile kök nedeni araştırmak istiyorsun.

Analiz planı

- Aylık geliri doğrula ve iptal/refund filtrelerini kontrol et
- LAG ile MoM değişimi hesapla
- Geliri country, channel, category gibi segmentlere ayır
- ROW_NUMBER/RANK ile düşüşe en fazla katkı yapan segmentleri bul
- Sipariş sayısı, AOV ve active customer bileşenlerini ayrıştır
- Cohort ve retention tarafında tekrar müşteri düşüşü var mı kontrol et
- Funnel eventlerinde conversion düşüşü var mı incele

Katkı analizi örneği

```
WITH m AS (
  SELECT DATE_TRUNC('month', order_date) AS month, channel,
         SUM(revenue) AS revenue
  FROM orders
  WHERE status='completed'
  GROUP BY 1,2
), x AS (
  SELECT month, channel, revenue,
         LAG(revenue) OVER (PARTITION BY channel ORDER BY month) AS prev_rev
  FROM m
)
SELECT month, channel, revenue, prev_rev, revenue-prev_rev AS delta,
       RANK() OVER (PARTITION BY month ORDER BY revenue-prev_rev) AS decline_rank
FROM x;
```

Yönetici çıktısı: "Gelir düştü" bir insight değildir. Örneğin: "Toplam düşüşün %68'i paid social kanalından geliyor; sipariş hacmi sabitken checkout→purchase dönüşümü 6.1 puan geriledi."

14. İleri SQL’de Debug ve Doğrulama

Her CTE’nin grain’ini yaz

Örnek: monthly_customer CTE’sinde 1 satır = 1 customer_id + 1 month. Bu cümleyi söyleyemiyorsan sorgunun grain’i belirsizdir.

Kontrol sorguları

```
SELECT COUNT(*) AS rows, COUNT(DISTINCT order_id) AS orders FROM ...;
SELECT MIN(date_col), MAX(date_col) FROM ...;
SELECT status, COUNT(*) FROM ... GROUP BY status;
SELECT customer_id, COUNT(*) FROM ... GROUP BY customer_id HAVING COUNT(*) > 1;
```

Window sonrası neden satır sayısı değişmemeli?

Sadece window function eklemek satır sayısını değiştirmez. Eğer değişiyorsa başka JOIN, WHERE veya DISTINCT etkisi vardır.

NULL ve sıfıra bölme

```
revenue / NULLIF(order_count, 0)
```

Boundary test

Cohort ve tarih sorgularını özellikle ayın ilk günü, son günü, yıl geçişi, leap year ve timezone sınırlarında test et.

15. Performans Mantığı - Analistin Bilmesi Gerekenler

Veri analisti DBA olmak zorunda değildir; ancak pahalı sorguların neden pahalı olduğunu anlayabilmelidir.

- SELECT * yerine gereken kolonları çek
- Filtreleri mümkün olduğunca erken uygula ama iş mantığını bozma
- Büyük tablolarda gereksiz DISTINCT kullanma
- JOIN anahtarlarının doğru olduğundan emin ol
- Fonksiyon uygulanmış kolonlarda indeks kullanımının değişebileceğini bil
- Window function'dan önce veri hacmini gereksiz yere büyütme
- Execution plan / query profile aracını öğren

Yanlış optimizasyon örneği

Sorgu daha hızlı olsun diye filtreyi erkene almak bazen sonucu değiştirir. Örneğin müşterinin "ilk siparişini" tüm tarihçeden bulman gerekiyorsa, önce son 90 güne filtreleyip sonra MIN(order_date) almak yanlış cohort üretir.

> Öncelik sırası: 1) Doğruluk, 2) okunabilirlik, 3) performans. Hızlı ama yanlış sorgu analitik değer üretmez.

16. İleri SQL Tasarım Desenleri

Problem	Tercih edilen desen
Her grubun son kaydı	ROW_NUMBER + PARTITION BY + ORDER BY DESC
Top N per group	RANK / DENSE_RANK
MoM / WoW değişim	LAG
Bir sonraki olay	LEAD
Kümülatif toplam	SUM OVER + frame
Hareketli ortalama	AVG OVER + ROWS frame
İlk aktivite	MIN + GROUP BY veya ROW_NUMBER
Cohort retention	first activity + activity month + cohort age
Funnel	event sequencing + conditional aggregation
Uzun sorgu	CTE zinciri + açık grain

QUALIFY notu

BigQuery ve Snowflake gibi sistemlerde window sonucunu filtrelemek için QUALIFY kullanılabilir. PostgreSQL'de genellikle subquery veya CTE gerekir.

```
-- BigQuery / Snowflake
SELECT *
FROM orders
QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY order_date DESC)=1;
```

17. Mülakat Tipi Sorular

- 1. ROW_NUMBER, RANK ve DENSE_RANK arasındaki fark nedir? Gerçek bir kullanım örneği ver.
- 2. Bir müşterinin ikinci siparişini nasıl bulursun?
- 3. Her kategori için en yüksek satış yapan ilk 3 ürünü nasıl getirirsin?
- 4. LAG ile aylık büyüme hesaplarken ilk ay için ne olur?
- 5. Window function ile GROUP BY arasındaki temel fark nedir?
- 6. CTE kullanmak sorguyu her zaman hızlandırır mı?
- 7. Cohort retention'da COUNT(*) neden yanlış sonuç verebilir?
- 8. Funnel analizinde yalnızca event varlığını kontrol etmek neden yetersizdir?
- 9. Running total ile moving average arasındaki fark nedir?
- 10. Bir query'de ROW_NUMBER sonrası rn=1 filtresini WHERE içinde aynı seviyede neden her DB'de yazamazsın?
- 11. Aynı order_date'e sahip iki siparişte "son sipariş" nasıl deterministik seçilir?
- 12. Timezone hatası funnel veya günlük KPI'ları nasıl etkiler?

Beklenen yaklaşım

Mülakatta yalnızca syntax söyleme. Önce grain'i, partition mantığını ve edge case'i açıkla; sonra SQL yaklaşımını ver. Özellikle duplicate, NULL, tie, zaman sırası ve eksik dönemleri belirtmek seni daha güçlü gösterir.

18. Uygulama Alıştırmaları

- 1. Her müşterinin ilk, ikinci ve son sipariş tarihini bul.
- 2. Her ay için gelir ve önceki aya göre yüzde değişimi hesapla.
- 3. Her kategori içinde ilk 5 ürünü revenue'ya göre sırala.
- 4. Bir müşterinin ardışık siparişleri arasındaki ortalama gün farkını hesapla.
- 5. Günlük gelirin 7 günlük hareketli ortalamasını üret.
- 6. Müşterileri ilk sipariş ayına göre cohort'a ata.
- 7. Cohort bazında month 0, 1, 2 ve 3 retention oranlarını hesapla.
- 8. product_view → add_to_cart → purchase funnel'ını kullanıcı bazında kur.
- 9. Purchase'ın view'dan en fazla 24 saat sonra gerçekleşmesi şartını ekle.
- 10. Aynı event'in duplicate kayıtlarını ROW_NUMBER ile temizle.
- 11. Ağustos gelir düşüşüne en çok katkı yapan ilk 3 channel'ı bul.
- 12. Her müşteri için lifetime revenue ve toplam içindeki yüzdesini hesapla.

Zorluk artırma

Her soruyu önce doğru sonucu üretmek için çöz. Ardından aynı soruya edge case ekle: ties, NULL, duplicate, missing month, refund, timezone veya session sınırı.

19. Modül Sonu Portfolio Projesi

Proje: E-Commerce Customer Lifecycle & Funnel Analysis

İstenen çıktılar

- Aylık revenue trendi + MoM growth
- Her müşterinin first/last order bilgisi
- Kategori bazlı Top-N ürün analizi
- Repeat purchase interval analizi
- Cohort retention matrisi için SQL çıktısı
- 4 adımlı funnel conversion analizi
- En büyük drop-off adımının segment bazlı root-cause analizi
- En az 3 iş önerisi ve bunları destekleyen sorgular

GitHub yapısı

```
sql-advanced-project/
  01_data_checks.sql
  02_window_functions.sql
  03_cohort_analysis.sql
  04_funnel_analysis.sql
  05_root_cause.sql
  README.md
```

README’de anlat

- Business problem
- Dataset grain ve tablo ilişkileri
- KPI tanımları
- SQL yaklaşımı
- Bulduğun insight’lar
- Varsayımlar ve veri kalitesi limitleri
- Business recommendations

20. Hızlı Referans - Cheat Sheet

```
-- Son kayıt
ROW_NUMBER() OVER (PARTITION BY key ORDER BY date DESC)

-- Sıralama
RANK() OVER (PARTITION BY grp ORDER BY metric DESC)

-- Önceki değer
LAG(metric) OVER (PARTITION BY grp ORDER BY date)

-- Sonraki değer
LEAD(metric) OVER (PARTITION BY grp ORDER BY date)

-- Kümülatif toplam
SUM(metric) OVER (ORDER BY date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)

-- Hareketli ortalama
AVG(metric) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)

-- Ay
DATE_TRUNC('month', date_col)

-- Güvenli bölme
num / NULLIF(den,0)
```

Bu modülü bitirdiğinde ne yapabilmelisin?

- Karmaşık sorguyu CTE’lerle anlamlı adımlara ayırabilmek
- Window function seçimini iş problemine göre yapabilmek
- Top-N, latest-record, running total ve growth sorgularını yazabilmek
- Cohort retention sorgusunu baştan sona kurabilmek
- Sıralı funnel mantığını SQL ile modelleyebilmek
- Sorgunun grain’ini, edge case’lerini ve veri kalitesi risklerini açıklayabilmek

Sonraki modül: İstatistik - mean/median/mode, variance/std, percentiles, distributions, sampling, correlation vs causation, confidence interval, hypothesis testing ve A/B testing.